

CS-2006: Operating Systems(Spring 2026)

Final Term Project

Groups

This is an individual project. You can make a group of 2-3 students and enter your group members' names in the sheet for later reference. Collaboration with other groups is not allowed.

Deadline: 5th May 2026

Please start your work on time, as the deadline will **not** be extended.

- **Project Deadline:** to be announced in class.
- **Late submissions**
 - o On 1–12 hours late, deduction of 10% marks.
 - o On 12–24 hours late, 20% marks deduction.
 - o Beyond 24 hours, the project will be marked zero.

Instructions

You are required to use only **C (or C++)** and the system calls and libraries studied in the OS labs (e.g. fork(), exec(), wait(), pipes (named/unnamed), the pthread library, mutexes, semaphores, signals, dup/dup2, etc.) to implement the scenario described below. The solution must be implemented and tested on Linux (preferably Ubuntu 22.04 or later) since the demo will be conducted on Ubuntu machines. Even if you develop on a different distribution or operating system, make sure your program runs as expected in an Ubuntu environment.

The program must compile cleanly with `gcc -Wall -Wextra -pthread` (or g++ equivalent) and produce no warnings. Hardcoding values that are supposed to be read from input files will result in a mark deduction.

Submission

All submissions must be uploaded on Google Classroom strictly before the deadline. Solutions sent on email will **not** be graded. To avoid last-minute problems (network issues, load shedding, Google Classroom being unavailable, etc.), you are strongly advised to start working from day

one. Each group will submit a single .zip file, named according to the roll numbers of the group members, containing the following:

- All .c / .cpp source files and a Makefile (or a small build script).
- The input.txt file you used during testing.
- A short README explaining how to compile and run.
- A project report covering the items listed below.

Project Report

The report should be 8–12 pages and must cover the following:

- **Title and Team Details:** project title and full names + roll numbers.
- **Problem Statement:** clearly define the problem you are solving in your own words.
- **System Design and Architecture:** describe how each component (data producer, worker pool, aggregator, reporter) is mapped to processes and threads. Explain your IPC and synchronization design.
- **Implementation Details:** explain how processes/threads are created dynamically, how input/output is handled, and how resources are released at the end.
- **Sample Output:** include screenshots or short tables showing the program in action the forward pipeline producing intermediate values, and the backward pass propagating signals.
- **Work Division:** describe what each team member contributed (be specific “did the report” is not enough).
- **Challenges Faced:** mention difficulties encountered and how you solved them.

Plagiarism Policy

Plagiarism is strictly prohibited. Any group submitting plagiarized work will receive zero. We will run code similarity tools and we also check against AI-generated code. The thresholds below apply to both inter-student plagiarism and copied code from the internet.

- Similarity in the range 21%–30%: 25% marks deduction.
- Similarity in the range 31%–50%: 50% marks deduction.
- Similarity above 50%: zero marks for the entire group.

Note: using AI tools to write your code is treated the same as copying. You may use AI to ask conceptual questions but not to generate the implementation. During the viva, every member of the group must be able to explain their own code line by line. If you cannot, you will not get the marks for that part even if the program runs correctly.

Parallel CSV Data Processing Pipeline

(Multi-Process, Multi-Threaded Data Engine on Linux)

Problem Statement

You are required to design and implement a parallel data processing pipeline on Linux that ingests CSV datasets, processes them through a pool of worker threads inside a worker process, aggregates the results, and produces a final report. The pipeline must be split across multiple **processes** that communicate using **named pipes (FIFOs)** and **shared memory**, while the worker process internally uses a pool of **POSIX threads** synchronized using **mutexes** and **semaphores**.

The objective is to demonstrate, in a single coherent project, every operating system concept covered during the course: process creation and lifecycle, signals, file descriptor manipulation, inter-process communication, multi-threading, synchronization, and shell scripting for orchestration.

Project Description

Modern data systems regularly process tabular data in CSV format sensor readings, retail transactions, click logs, and so on. A naïve single-threaded program is slow because it cannot use the multiple cores available on a modern machine. The aim of this project is for you to build a small but realistic pipeline that uses the operating system primitives directly, without depending on high-level frameworks.

The pipeline is composed of **four cooperating programs**, each compiled into its own executable. They are launched and coordinated by a master Bash script. Each program is responsible for one stage of the pipeline:

- **dispatcher** : the master process. Reads the configuration, creates the FIFO and shared-memory segment, forks and execs the other three programs, installs signal handlers, and waits for them to finish.
- **ingester** : reads CSV files from an input directory, splits each file into chunks, and writes those chunks into a named pipe.
- **processor** : reads chunks from the pipe and feeds them into a thread pool. Each worker thread parses CSV rows and updates a shared aggregation table that is protected by a mutex. The final aggregated result is written into a POSIX shared-memory segment.
- **reporter** : reads the shared-memory segment and writes a human-readable report and a machine-readable CSV summary to disk.

User Inputs and Configuration

The pipeline is configured at runtime through command-line arguments to the master Bash script:

- **Input directory** : the directory that contains the CSV files to process.
- **Output directory** : where the report and CSV summary will be written.
- **Number of worker threads (N)** : how many threads the processor will spawn. Must be configurable, not hardcoded.
- **Queue size (Q)** : the size of the bounded buffer between the FIFO reader and the worker threads. Must also be configurable.

Your program must **not** hardcode the number of CSV files, the number of columns, or the queue/thread counts. A different input directory will be used during the demo.

System Design and Architecture

The data flow at a high level is straightforward. The dispatcher creates the FIFO and the shared-memory segment first, then spawns the three child processes using `fork()` and replaces their image using one of the `exec*()` calls. After spawning, the dispatcher only handles signals and waits for the children to terminate. The ingester writes CSV chunks into the FIFO; the processor reads them, runs them through its thread pool, and places the aggregated result in shared memory; the reporter reads the shared-memory segment and writes the final output files. Control signals (SIGTERM during shutdown, SIGUSR1 for status dumps) flow back from the dispatcher to the children.

The mapping of OS concepts onto the components is as follows:

Concept	Where it appears in the project
<code>fork()</code>	dispatcher creates ingester, processor, reporter as child processes.
exec family	Each forked child calls <code>execvp()</code> so it becomes a real, separate executable.
<code>wait / waitpid</code>	dispatcher reaps every child and records its exit status. No zombies are allowed.
<code>exit()</code>	Every component exits with a defined status code (see “Exit Codes” below).
<code>getpid / getppid</code>	Every log line printed by every component must carry its own PID and its parent’s PID.
<code>pthread_create / join</code>	processor runs N worker threads in a thread pool.

Concept	Where it appears in the project
Thread attributes	Worker threads must be created with an explicit <code>pthread_attr_t</code> — stack size and detach state set by your group, and explained in the report.
Mutex	Protects the shared aggregation table inside the processor.
Semaphores	Bounded-buffer (empty / full) semaphores between the FIFO reader and the worker threads. A second named POSIX semaphore signals the reporter when aggregation is done.
Named pipe (FIFO)	ingester → processor data link. Created using <code>mkfifo()</code> .
IPC — shared memory	processor → reporter result handoff. Created using <code>shm_open()</code> + <code>mmap()</code> .
dup / dup2	Each child redirects its stdout/stderr to a per-process log file using <code>dup2()</code> before <code>exec</code> . At least one component must also use <code>dup()</code> to save and restore a file descriptor.
Signals	<code>SIGINT</code> and <code>SIGTERM</code> cause graceful shutdown. <code>SIGCHLD</code> is handled by the dispatcher to reap children. <code>SIGUSR1</code> dumps live statistics to stderr.
Shell scripting	A <code>run.sh</code> script builds the project, launches the dispatcher, and produces a summary on exit. Must use functions, <code>getopts</code> , and a trap for cleanup.

Component Specifications

dispatcher

- Parses command-line arguments: input directory, output directory, N (threads), FIFO path, shared-memory name.
- Creates the FIFO using `mkfifo()` and the shared-memory segment using `shm_open() + ftruncate() + mmap()`.
- Installs signal handlers for `SIGINT`, `SIGTERM`, `SIGCHLD` and `SIGUSR1`.
- Forks three children. Each child calls `dup2()` to redirect `stdout/stderr` into a per-process log file under the `logs/` directory before calling `execvp()`.
- Blocks in a `sigsuspend()` loop (**not** a busy-wait) until all children exit or until a shutdown signal arrives.
- On shutdown: forwards `SIGTERM` to the children, calls `waitpid()` for each one, unlinks the FIFO, unmaps and unlinks the shared-memory segment, and removes the named semaphore.
- Prints a final summary line per child: PID, exit status, and runtime in seconds.

ingester

- Scans the input directory for `*.csv` files.
- Opens the FIFO for writing. (The `open()` call will block until the processor opens its end for reading this is the intended synchronization.)
- Reads each CSV in chunks (e.g. 64 KB or 1000 rows). Prepends a small binary header to each chunk containing `chunk_id`, `byte_count`, and `source_file_id`, then writes the chunk into the FIFO.
- On `SIGTERM`, writes a special EOF chunk, closes the FIFO, and exits cleanly.
- On `SIGUSR1`, prints to `stderr`: number of files processed, chunks sent, total bytes sent — and then continues working.

processor

- Opens the FIFO for reading.
- Creates a thread pool of N worker threads using `pthread_create()` with an explicit `pthread_attr_t`.
- Maintains a bounded queue of size Q. A single dedicated **reader thread** pulls chunks out of the FIFO and enqueues them; the N worker threads dequeue and process.

- The queue is protected by a mutex. The producer-consumer relationship between the reader and the workers is coordinated using two semaphores: `sem_empty` (initialised to Q) and `sem_full` (initialised to 0).
- Each worker parses CSV rows from its chunk and updates a shared aggregation table (e.g. a hash table keyed by category). The aggregation table itself is guarded by its own mutex so updates are atomic.
- When the EOF chunk is received, the reader thread enqueues N “poison pill” items so every worker exits cleanly. All workers are then `pthread_join()`ed by the main thread.
- After all workers finish, the aggregation table is serialised into the shared-memory segment, and a named POSIX semaphore is `sem_post()`ed so that the reporter knows the data is ready.

reporter

- Opens the shared-memory segment and the named semaphore created by the processor.
- Calls `sem_wait()` on the named semaphore. **Do not poll.**
- Reads the aggregation table from shared memory and produces two output files: `report.txt` (human-readable) and `report.csv` (machine-readable) in the output directory.
- Before writing the human-readable file, demonstrate the use of `dup()` to save the current `stdout`, `dup2()` to redirect `stdout` to the report file, write the report using normal `printf()`, and then restore `stdout` from the saved descriptor. This block must be commented in the source code.
- Sends `SIGUSR1` back to the dispatcher (PID obtained via `getppid()`) to indicate the report is ready.
- Exits with status 0 on success, non-zero on failure.

run.sh (orchestrator)

- Accepts options via `getopts`: `-i` (input dir), `-o` (output dir), `-n` (number of threads), `-c` (clean), `-h` (help).
- Verifies that `gcc` and `make` are installed and that the input directory contains at least one `.csv` file.
- Builds the project by invoking `make`. If `make` fails, prints a clear error message and exits non-zero.
- Launches the dispatcher and writes its PID to a `.pid` file.
- Installs a Bash trap on `EXIT/INT/TERM` that kills any background process the script started and removes the `.pid` file.

- After the dispatcher exits, prints a summary: total runtime, number of records processed (read from report.csv), and the final exit status.
- Must use at least three Bash functions, one loop, one case statement, and one arithmetic expansion. Must pass shellcheck with no warnings.

Standard Exit Codes

Use the following exit codes consistently across all four programs:

Code	Meaning
0	Success. All components finished cleanly.
10	Bad command-line arguments.
20	Failed to create FIFO, shared memory, or semaphore.
30	A child process died unexpectedly (SIGSEGV, SIGKILL, etc.).
40	I/O error (missing input files, unreadable CSV, etc.).
130	Interrupted by SIGINT (Ctrl-C). Clean shutdown succeeded.
143	Interrupted by SIGTERM. Clean shutdown succeeded.

Implementation Details

- Your program must print appropriate status messages during execution showing the progress of each stage and the final result.
- The number of worker threads, the queue size, and the input/output directories must be configurable via command-line arguments not hardcoded.
- The CSV input format must be read from input.txt (or the files placed in the input directory). It must **not** be embedded in the source code.
- The result must be written to report.txt and report.csv in the configured output directory.
- Close all files and pipes after use, destroy the semaphores/mutexes, unmap and unlink the shared memory, and remove the FIFO using unlink() before the program exits.
- Pressing Ctrl-C while the pipeline is running must not leave behind any zombie processes, stale FIFOs in /tmp, or leaked shared memory segments. After your program finishes, the output of `ipcs -m` and `ls /tmp/<your-fifo>` must show nothing belonging to your project.

Input File Format

Your program must follow a predefined input format. The data is provided in CSV form one record per line, comma-separated. The first column represents the key (category, sensor ID, user ID, or symbol depending on the assigned dataset variant), while the remaining columns contain numeric fields to be aggregated. Field counts may vary between files, but each file remains internally consistent.

Dataset Variants

The aggregation logic depends on the dataset variant assigned. The four possible variants are:

- **Retail transactions:** per-category total revenue and top-N products
- **IoT sensor logs:** per-device moving averages and anomaly counts
- **Web clickstream:** per-user session length and bounce rate
- **Financial tick data:** per-symbol VWAP, high, low, and total volume

Each student will be assigned one variant during the project announcement. You are only required to implement the logic for your assigned variant. The aggregation logic is intentionally simple, as the focus of this project is on operating systems concepts rather than data analytics.

Submission Structure:

```
OS_FinalProject_<Section>/
├── Makefile           # builds all four executables
├── run.sh             # master orchestration script
├── README.md          # quickstart: how to build & run
├── report/
│   ├── report.pdf     # full written report (max 15 pages)
│   └── architecture.png # architecture diagram
├── src/
│   ├── dispatcher.c
│   ├── ingester.c
│   ├── processor.c
│   ├── reporter.c
│   └── common/         # shared headers & helpers
├── data/
│   └── sample.csv      # small sample, no massive files in archive
├── logs/              # empty directory, created by run.sh
└── DECLARATION.txt    # signed academic honesty statement
```

Project Rubric

Component	Marks
Understanding of Problem & Viva	60
Clear explanation of process/thread mapping and OS-level reasoning	20
Accurate answers to questions about OS concepts during viva	40
Design & Architecture	45
Correct mapping of components to processes/threads	10
Dynamic creation of processes/threads at runtime from user input	15
Inter-process communication design and overall workflow	10
Resource lifecycle management (FIFO and shm creation, closing, unlink, cleanup)	10
Forward / Pipeline Implementation	30
Correct CSV chunking and FIFO transfer between ingester and processor	10
Correct thread pool with bounded queue and proper synchronization	10
Correct aggregation and shared-memory handoff to the reporter	10
Backward Signal & Reporting	20
Reporter waits on the named semaphore (no busy-wait)	5
Correct dup()/dup2() demonstration when writing the human-readable report	5
Reporter signals the dispatcher when the report is ready	5
Final report.txt and report.csv are correctly produced	5
Inter-Process Communication	15
Correct use of named pipes for chunk transfer	5
Correct use of POSIX shared memory and named semaphore	5
Proper cleanup of all IPC resources (no leftover ipcs entries)	5
Concurrency Control	15
Proper use of mutex(es) to avoid race conditions	8
Proper use of semaphores for the bounded buffer	7

Component	Marks
Input/Output and Shell Script	15
Reads input from input directory, writes report.txt and report.csv	8
run.sh works correctly, passes shellcheck, handles errors gracefully	7
Status Messages, Robustness, and Cleanup	10
Clear log messages with PID/PPID for every component	5
Ctrl-C produces a clean shutdown with no leaked resources	5
Project Report	20
TOTAL	230

Note on viva: Marks are awarded individually based on the viva. If a student cannot explain the part of the code they claim to have written, they will receive zero for that component even if the program runs correctly. The viva is the single most important component of the grade please prepare for it.